GraphWeave — Master Specification

GraphRAG for Multi-Hop Questions

Version 1.0 | 27 October 2026 | Engineering specification for an independently buildable portfolio project

Document control: Treat this document as the project-specific source of truth. If implementation choices change, update the relevant decision record and tests.

Product overview

Vector-only RAG can miss answers whose evidence is distributed across documents and connected by entity relationships, such as who acquired a company that owns a product mentioned in another source.

Objectives and non-goals

- Build a GraphRAG system that extracts entities and typed relationships from source documents, maintains provenance, combines graph traversal with vector retrieval, and evaluates multi-hop answer quality against a vector-only baseline.
- Deliver a runnable local development environment with deterministic setup instructions and sample data.
- Provide API contracts, automated tests, security controls, operational telemetry, and reproducible evaluation evidence.
- Keep external services behind adapters so providers can be replaced without rewriting business logic.
- Non-goal: claiming production certification, guaranteed savings/accuracy, or enterprise compliance without measured evidence.

Architecture and technology

- Frontend: React + TypeScript + Vite + Tailwind CSS; typed API client; accessible loading, empty, error, and permission-denied states.
- Backend: Python + FastAPI + Pydantic; layered modules for API, services, repositories, domain models, adapters, and configuration.
- Persistence: PostgreSQL with migrations; pgvector for embeddings; Redis where caching/queues are needed.
- LLM/embedding providers: interfaces plus at least one configurable implementation and a deterministic mock provider for tests.
- Deployment: Docker Compose for local services; health checks; .env.example containing names and safe placeholders only.
- Testing: Pytest for unit/integration tests; frontend component tests; API contract tests; end-to-end smoke tests.

Functional requirements

- Ingestion and provenance: Parse documents, preserve document/version/page/section provenance, chunk content, and maintain deterministic IDs so graph facts can be traced back to source evidence.
- Entity and relation extraction: Use schema-constrained LLM extraction with validated entity types, relation types, confidence, source chunk IDs, and extraction model/version. Treat extracted facts as candidates, not unquestionable truth.
- Entity resolution: Normalize names and aliases, merge only with explicit confidence rules, preserve aliases, and support review of ambiguous matches. Avoid merging people or organizations solely because names are similar.

- Knowledge graph: Store entities and typed edges in Neo4j. Include tenant or corpus scope, source chunk references, timestamps, confidence, and extraction provenance on relationships.
- Hybrid retrieval planner: Retrieve seed chunks/entities by vector similarity, expand graph neighborhoods with bounded depth/fan-out, optionally use relation/path queries, and fuse graph and vector evidence with deduplication.
- Answer synthesis: Build a compact evidence bundle containing source snippets and relationship paths. Generate answers with citations and distinguish directly stated facts from inferred connections.
- Graph inspection UI: Allow users to inspect entities, relationships, source provenance, and the paths used for an answer. Scope all graph queries to the authorized corpus/tenant.
- Evaluation harness: Compare vector-only, graph-only, and hybrid retrieval on a curated multi-hop dataset. Measure answer correctness, evidence recall, citation precision, latency, and token cost.

Data model

Neo4j nodes: Entity, Document, Chunk; relationships: MENTIONS, RELATES_TO, ALIAS_OF, DERIVED_FROM (or equivalent typed relations). PostgreSQL: documents, chunks, ingestion_jobs, extraction_runs, entity_aliases, evaluation_questions, evaluation_runs, request_traces. Every graph fact must preserve source provenance and scope.

API contract baseline

- GET /health/live — process liveness; GET /health/ready — dependency readiness.
- POST /api/v1/documents — ingest a document or submit an ingestion job.
- GET /api/v1/documents — list only documents visible to the authenticated user/scope.
- POST /api/v1/query — run retrieval and answer generation; return answer, citations/evidence, trace ID, and safe diagnostic metadata.
- GET /api/v1/metrics/summary — return dashboard aggregates for authorized users.
- GET /api/v1/requests/{request_id} — fetch permitted request diagnostics without exposing secrets or unauthorized content.
- Use Pydantic request/response schemas, pagination, consistent error envelopes, request IDs, and explicit API versioning.

Security, privacy, and reliability

- Validate all inputs; enforce authentication and authorization on every protected route; apply rate limits and request-size limits.
- Use least privilege for database/service accounts; parameterized queries; safe CORS; secure headers; no secrets in source control or logs.
- Define retention and deletion behavior for documents, embeddings, cache entries, graph facts, and telemetry.
- Treat documents and retrieved text as untrusted data; defend against prompt injection and instructions embedded in source material.
- Use timeouts, bounded retries with backoff, circuit breakers or provider fallback where appropriate, and graceful degradation.
- Log identifiers and operational metadata rather than raw sensitive content by default.

Observability and evaluation

- Emit structured logs with correlation/request ID and component-level timings.
- Measure request volume, errors, retrieval latency, generation latency, total p50/p95 latency, token usage when available, and estimated cost.
- Maintain a versioned evaluation dataset with expected answer/evidence and documented scoring rules.

- Report quality together with performance/cost; do not present latency or cost improvements without a comparable baseline.
- Store model, prompt, embedding, corpus/index, and policy versions with each evaluation/request trace.

Acceptance criteria

- Entity and relation extraction uses a documented schema and validates outputs before graph writes.
- Every graph relationship used in an answer can be traced to source chunk(s) and document version(s).
- Hybrid retrieval returns evidence from multiple connected documents for designated multi-hop questions.
- Vector-only and hybrid baselines are evaluated on the same dataset and prompt/model settings.
- Answer responses include citations and identify uncertainty or missing evidence instead of fabricating a path.
- Graph traversal is bounded by depth, fan-out, timeout, and tenant/corpus scope.
- Entity resolution preserves aliases and avoids unsafe automatic merges when confidence is low.
- UI exposes graph paths and provenance while enforcing the same access policies as document retrieval.

Risks and mitigations

- LLM extraction can create false relationships; preserve provenance, confidence, validation, and evaluation.
- Entity resolution can merge distinct entities; ambiguous matches should remain separate or require review.
- Graph expansion can explode in size; enforce traversal depth, fan-out, timeouts, and query budgets.
- Graph answers can sound certain despite weak paths; require evidence-backed citations and explicit uncertainty.

Repository structure

frontend/
backend/app/{api,core,domain,services,repositories,adapters,schemas}/
backend/tests/{unit,integration,security,evaluation}/
migrations/
infra/docker-compose.yml
scripts/
data/sample/
docs/{architecture,api,threat-model,evaluation,runbook}/
.env.example
README.md

Definition of done

- A new developer can follow README instructions to start the app from a clean checkout.
- Database migrations and seed data run reproducibly.
- Unit, integration, security-critical, and frontend build tests pass.
- No secrets are committed; .env.example contains only placeholders.
- API docs, architecture notes, threat model, test results, known limitations, and demo instructions are present.
- The final report distinguishes implemented features from planned work and measured results from estimates.

Product-specific design decisions

- Graph facts are candidate claims with provenance, not ground truth. Preserve source chunk IDs, document versions, extraction model/version, and confidence.

- Graph traversals are bounded by maximum depth, fan-out, result count, and time budget; all traversals must obey corpus/tenant scope.
- Use conservative entity resolution; preserve aliases and queue ambiguous matches for review rather than forcing merges.
- Hybrid retrieval must return the evidence path and source snippets used to answer a multi-hop question.
- Evaluation compares vector-only, graph-only, and hybrid retrieval using identical questions and controlled model/prompt settings.